# SpringForge

**Intelligent Spring Boot Development Ecosystem**

# Spring Forge

## The

Developers waste 50-70% of time on manual architecture setup, boilerplate coding, quality checks, and CI/CD configuration instead of building features.

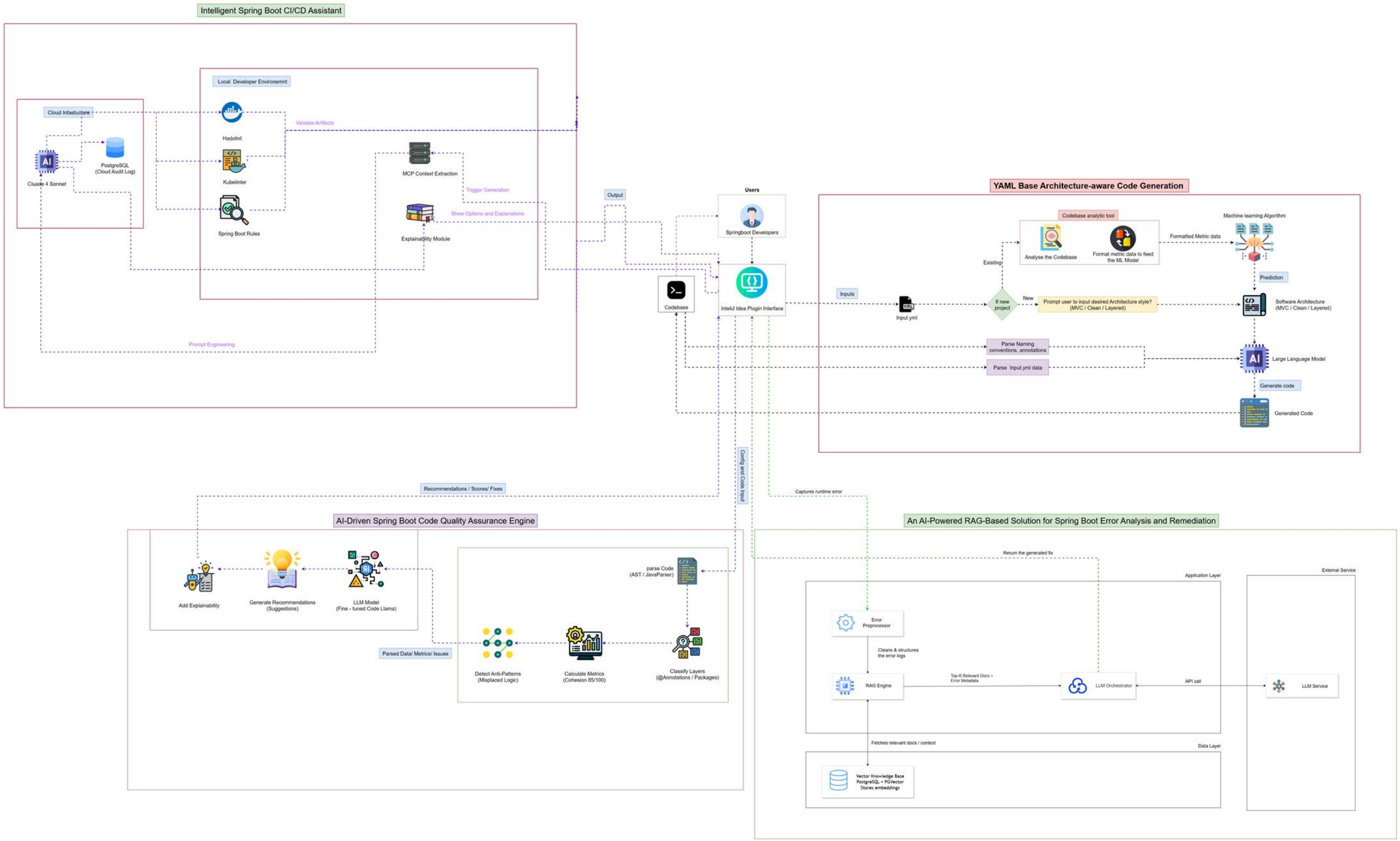
## Impact/

- Time Waste
- Quality Issues
- Deployment Delays
- Cost

## Benefits

- Springboot developers

# Overall System Diagram



# IT22076052

Mahindarathna D.G.P.T.D

Software Engineering

25-26J-
451

# ARCHITECTURE-AWARE Spring Boot Code Generation Plugin

# Architecture-Aware Spring Boot Code Generation Plugin

## The Sub

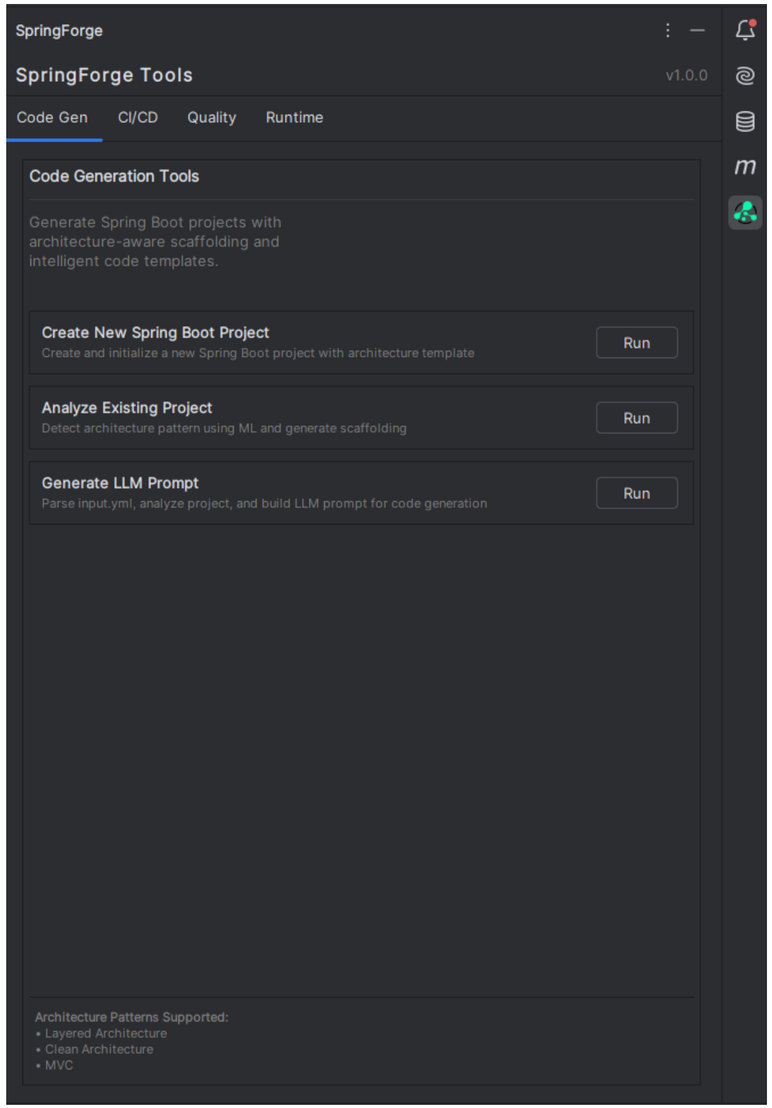
Not Having a proper scaffolding tool for Spring Boot and Generic code generators ignore project-specific architecture leading to inconsistent code.

## How solution will

- Addresses the problem of manual, inconsistent Spring Boot project scaffolding
- Automatically analyzes existing project structure and detects architecture (Layered, MVC, Clean) using a trained machine learning model.
- The plugin analyzes the active project, extracts specific naming conventions and folder structures, and combines them with user input and predicted architecture to generate proper codes.
- Reduces setup time and architectural mistakes during early development

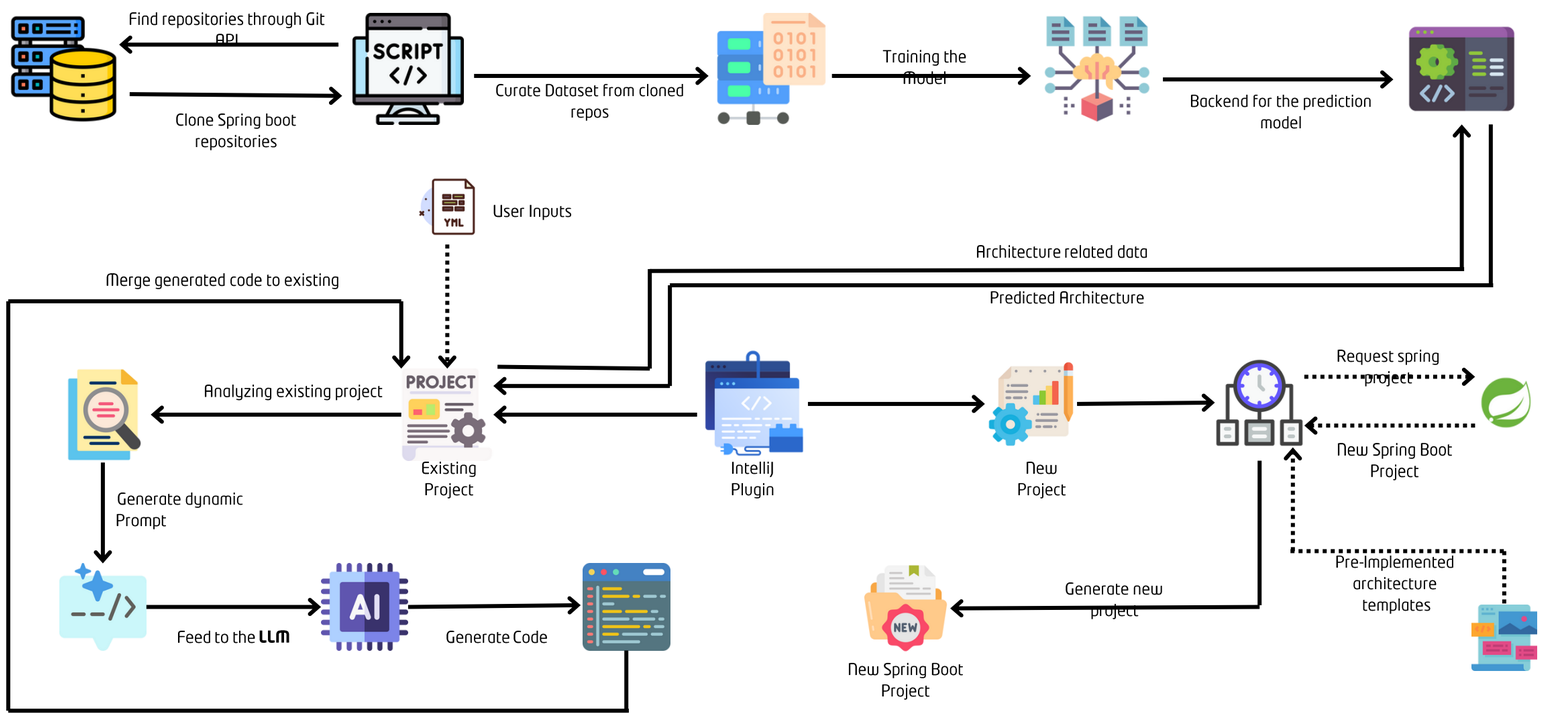
## User Requirements

- Eliminates manual boilerplate setup by integrating a custom Spring Initializr wrapper directly in the IDE to create projects with pre-defined architectural structures (Layered/Clean or MVC)
- Maintain consistent architecture and naming conventions
- Workflow Integration: Fulfills the requirement for an IDE-native experience; users analyze projects and generate code without leaving IntelliJ.



# Architecture-Aware Spring Boot Code Generation Plugin

## Design Excellence




- IntelliJ Plugin–based solution integrated directly into developer workflow.
- Hybrid Analysis Engine: A novel contribution combining Static Analysis to extract code features with Machine Learning to classify the project architecture before generating code.
- Dynamic Prompt Construction: Unlike standard tools that use static templates, SpringForge dynamically builds prompts injecting live project constraints (package depth, annotation usage, software architecture) to ensure LLM output is structurally valid.
- Decoupled Microservice design where the heavy ML processing is offloaded to a Cloud/FastAPI service, keeping the IntelliJ Plugin lightweight and responsive.

# IT22562524

Ariyasena M.K.M.M.
Software Engineering
25-26J-
451

# An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

# An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

## Problem

Developers waste 30-40% of debugging time context-switching between IDE, Stack Overflow, and documentation. Generic search returns irrelevant results.

## User

IDE-integrated RAG system that provides AI-powered debugging assistance without leaving the development environment.

## Functional

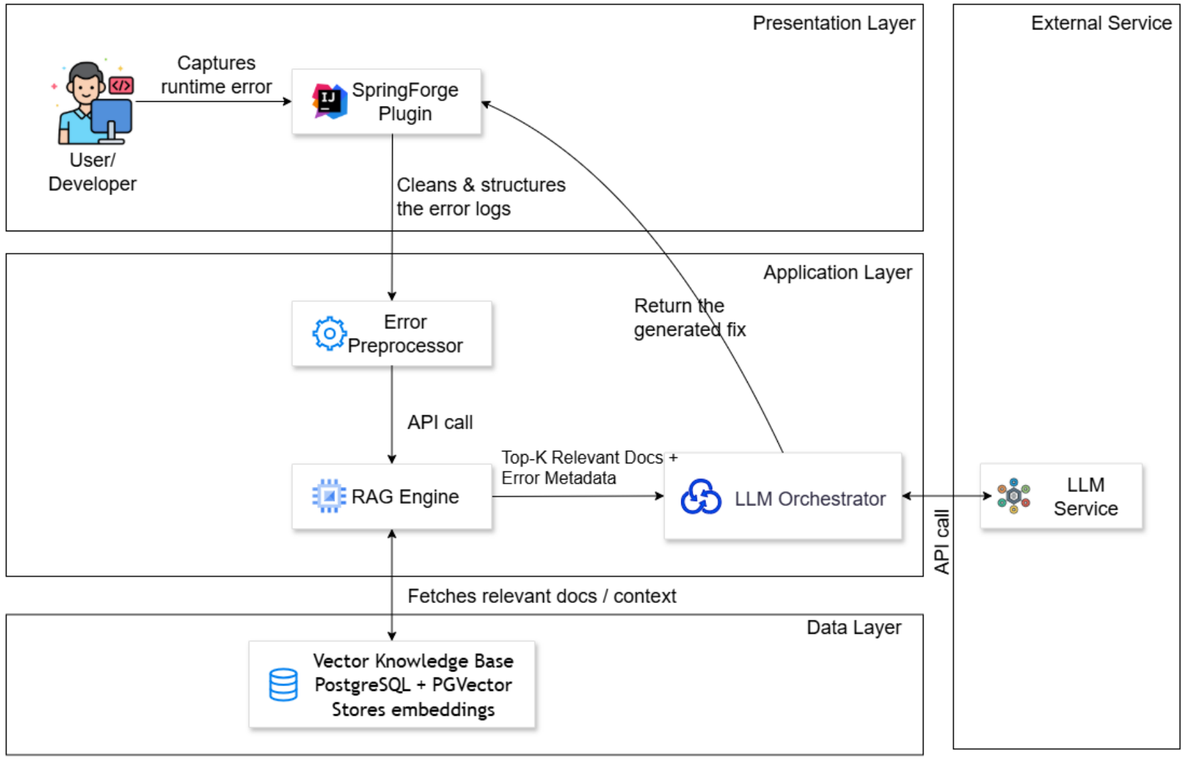
- Plugin should be able to provide tool window for runtime error input.
- Plugin should be able to automatically extract relevant code files.
- Plugin should support cosine similarity search on a vector database to retrieve semantically relevant entries.
- Plugin should display AI-generated responses clearly within the tool window.

.

- Plugin should be able to process requests and return results within 10 seconds.
- Plugin should be able to achieve >75% precision in retrieving relevant documentation
- Plugin should be able to generate correct fixes with >90% accuracy

# An AI-Powered RAG-Based Solution for Spring Boot Runtime Error Analysis and Remediation

## Design Excellence




**Novel Contribution: AI powered RAG-based Spring Boot debugger with IDE integration**

| Design Aspect | Innovation | Impact/Result |
|---|---|---|
| Semantic understanding | Vector embeddings | 80% precision |
| Grounded AI | RAG module | Reduces hallucination |
| Performance optimization | Lightweight model + async pipeline | 8.8s response |

# IT22060662

Jameela M.J.F.

Software Engineering

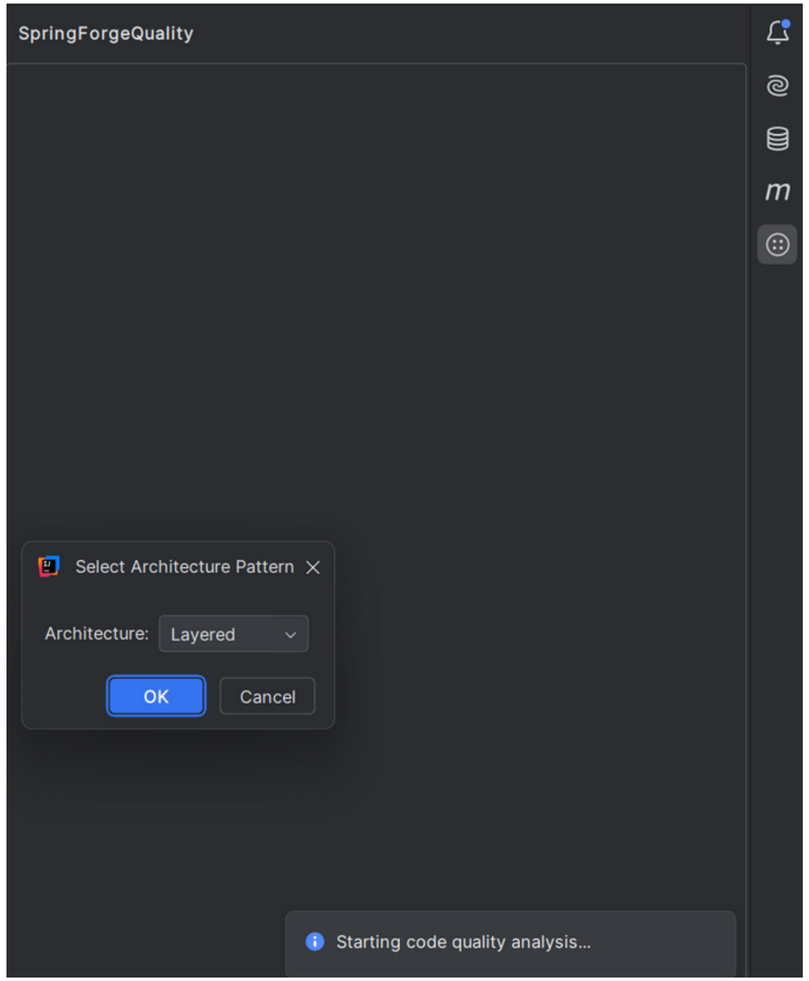
25-26J-451

# AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

# AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

## The Sub Problem

Spring Boot developers lack tools to automatically detect architecture violations and measure code quality. Manual reviews miss hidden anti-patterns, causing inconsistent structure, poor maintainability, and increased debugging effort within large or evolving projects.




## How solution will

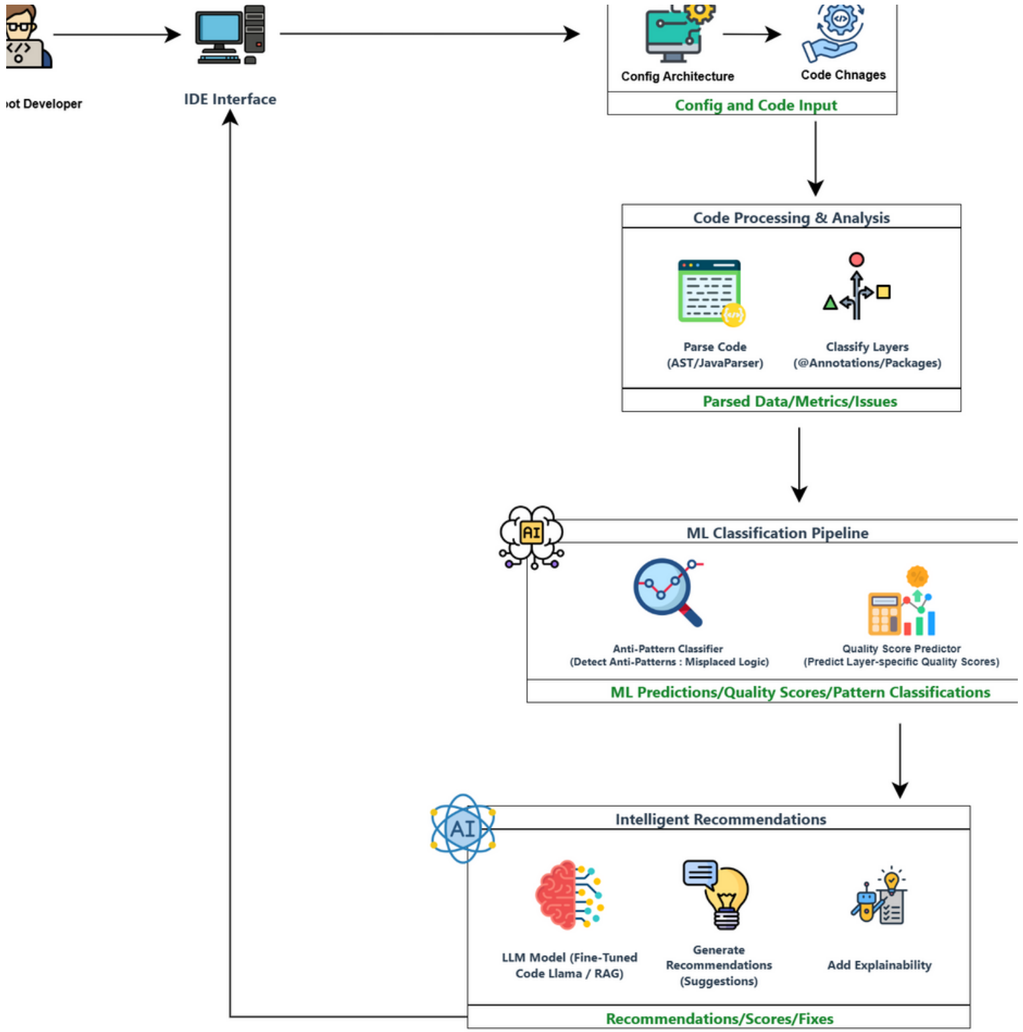
- IntelliJ plugin automatically analyzes the current Spring Boot project.
- Uses a trained ML model to detect architecture issues.
- Uses a second ML model to generate a 0–100 code quality score based on architecture rules.
- Automatically extracts project features (lines of code, imports, annotations, dependencies, complexity, layers) using IntelliJ PSI.
- Shows instant feedback inside IntelliJ—no external tools needed.
- Provides AI-generated explanations and recommended fixes to improve architecture and code quality.

## User Requirements

- Generates a clear 0–100 code quality score, helping teams measure structural health and maintain long-term code consistency.
- Produces simple explanations and recommended corrections, enabling faster debugging and more maintainable code.
- Provides all analysis, warnings, and reports inside IntelliJ's tool window—no need to switch applications or use external scanners.

# AI-Driven Code Quality Spring Boot Assurance Engine As A Plugin

## Design Excellence




**1. Dual-Engine AI Architecture**

- Anti-Pattern Classifier + Quality Score Regression
- Detects violations and quantifies code quality (0–100)

**2. IntelliJ-Native Smart Analysis**

- Uses PSI-based static analysis
- Real-time extraction of LOC, dependencies, annotations, complexity
- Results shown instantly inside a custom tool window

**3. Architecture-Aware Intelligence**

- Understands Layered/Hexagonal/Clean/MVC constraints
- Detects context-specific issues (business logic placement, missing transactions, tight coupling, etc.)

**4. Actionable Developer Feedback**

- Explains why the issue happened and how to fix it
- Makes the ML output understandable and immediately usable

**5. Deployment-Ready ML Service**

- Clean FastAPI microservice with schema validation
- Model, encoder, and feature mapping fully encapsulated

# IT22056320

Senevirathna K.M.U.T

Software Engineering

25-26J-451

# Intelligent Spring Boot CI/CD Assistant

# AI-Powered Infrastructure Generation with Security Validation and Explainability

## The Sub

Lack of secure, Spring Boot-aware CI/CD artifact generation with existing tools producing insecure infrastructure code lacking validation and explainability.

## How solution will

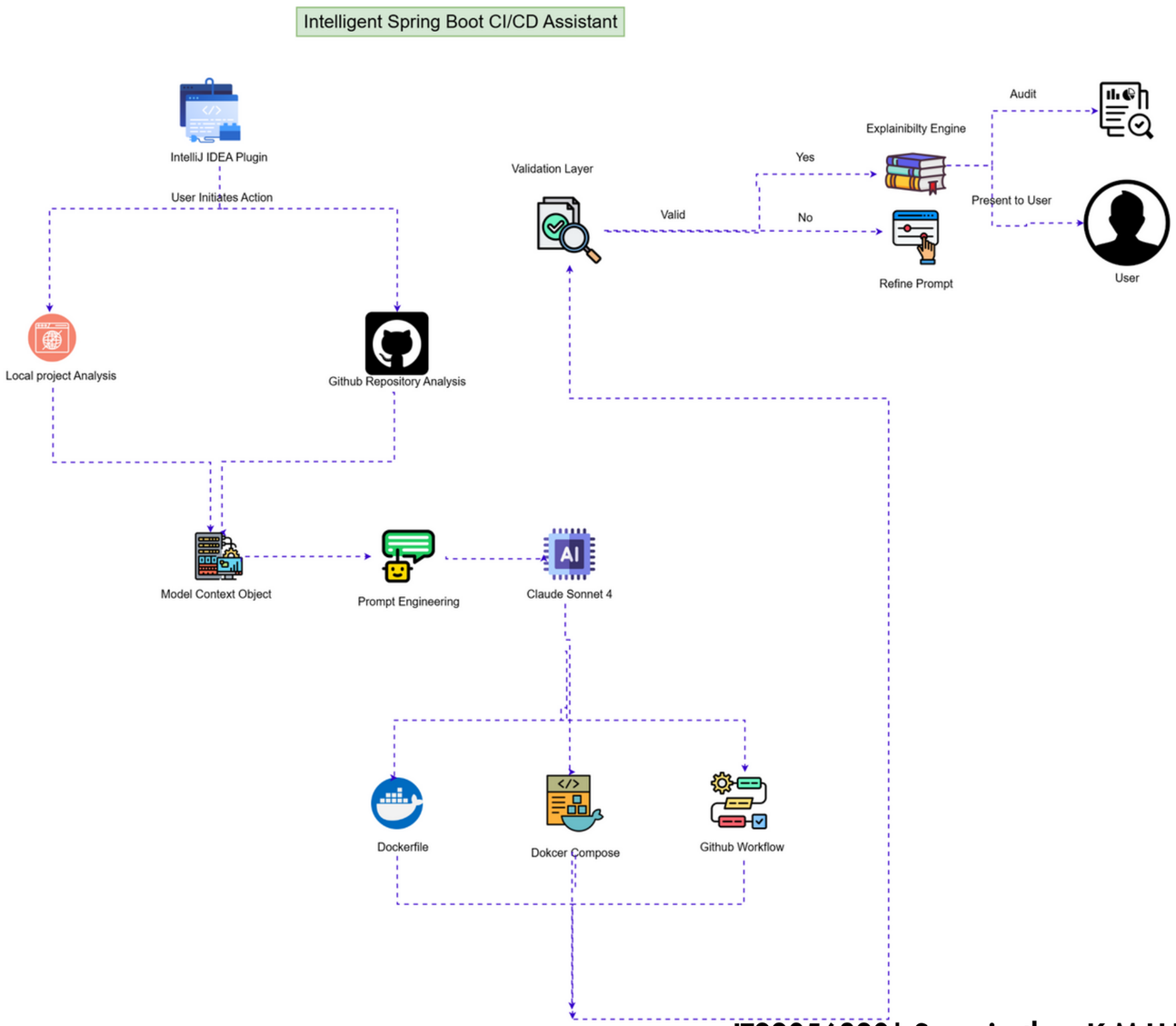
- Addresses the problem of manual, error-prone, and insecure DevOps artifact creation for Spring Boot projects
- Automatically analyzes project context (dependencies, build tools, configurations) using Model Context Protocol (MCP)
- Generates secure, production-ready deployment artifacts (Dockerfiles, Docker Compose, GitHub Actions) with integrated multi-layer validation (Hadolint, KubeLinter, Spring Boot rules) and line-by-line explainability for DevSecOps education
- Reduces deployment configuration time by ≥50% and eliminates common security vulnerabilities

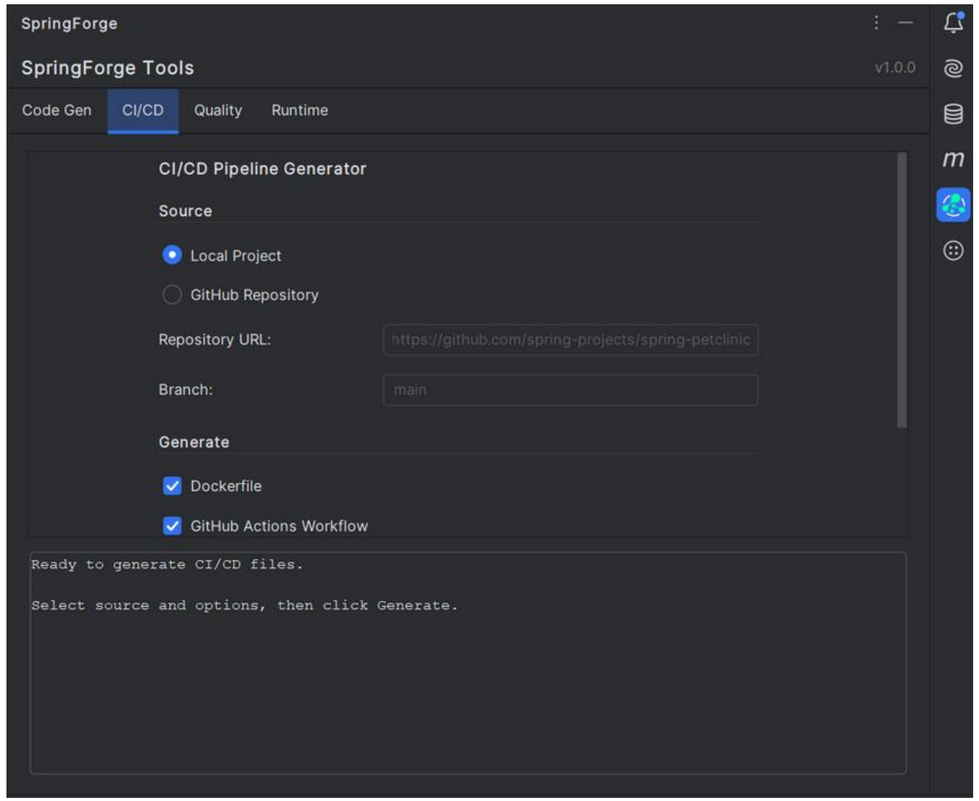
## User Requirements

- Ability to generate deployment artifacts without manual DevOps expertise
- Automated security validation preventing hardcoded secrets, root-level containers, and missing resource limits
- Context-aware generation matching specific project requirements (Java version, dependencies, ports)
- Educational explanations for junior developers learning containerization and CI/CD
- Seamless integration inside IntelliJ IDEA with audit logging for compliance and traceability

# AI-Powered Infrastructure Generation with Security Validation and Explainability

## Design



# Commercialization Strategy

## Customer

- Spring Boot developers struggling with architecture decisions, code quality, and DevOps setup
- Junior Developers & Newcomers

## Market

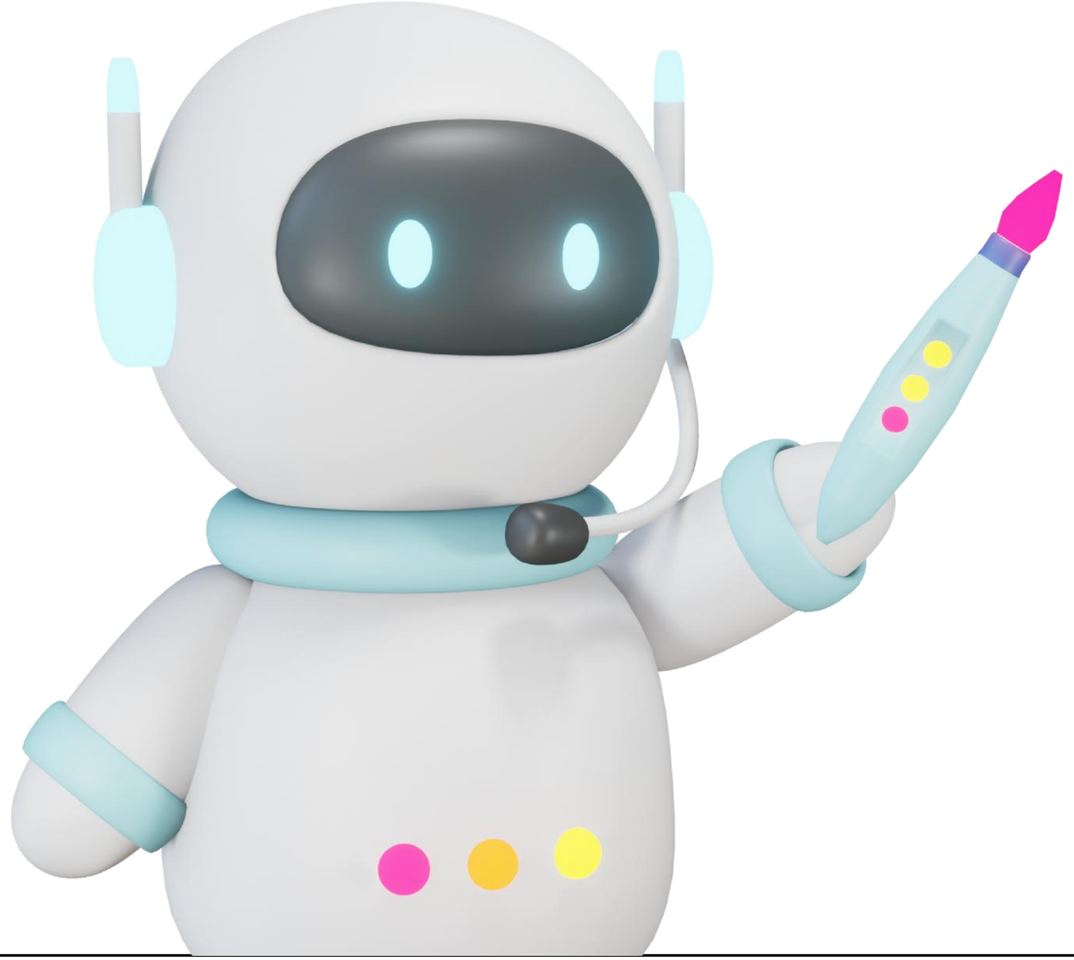
- Out of 8.7M Spring Boot developers, we're targeting around 10%.

## Unique

- AI-Powered Architecture Detection
- Dual-Source Analysis
- End-to-End Automation
- Context-Aware Intelligence
- Zero-Vendor Lock-in

## Revenue & Cost

- Revenue Model: Freemium Subscription.
- Community: Free
- Professional and Enterprise editions with paid subscription with enhanced features



# Thank You